

Design Pattern Documentation

Observer Design Pattern – Product Availability Notification System

1. Project Title

Product Availability Notification System using Observer Design Pattern

2. Design Pattern Used

Observer Pattern

Definition

The Observer Pattern is a behavioral design pattern where one object (called the Subject) maintains a list of dependent objects (Observers) and automatically notifies them whenever its state changes.

In this project:

- **Product** acts as the Subject.
- **User** acts as the Observer.

When the product availability changes, all subscribed users receive updates automatically.

3. Problem Statement

In many shopping systems, users want to know when a product becomes available or unavailable.

Without the Observer Pattern:

- The user must manually check the product status repeatedly.
- The system becomes inefficient and tightly coupled.

Using the Observer Pattern:

- Users subscribe to products.
 - Products automatically notify users whenever availability changes.
-

4. Objectives

- Notify users automatically when product availability changes.
 - Reduce tight coupling between classes.
 - Apply the Observer Design Pattern correctly.
 - Improve system flexibility and maintainability.
-

5. UML Class Description

Subject (Interface)

Represents the publisher object responsible for managing observers.

Methods

- subscribe(observer : Observer)
Adds an observer to the list.
 - unsubscribe(observer : Observer)
Removes an observer from the list.
 - notifyObservers()
Sends updates to all subscribed observers.
-

Product (Concrete Subject)

Represents a product whose availability status can change.

Attributes

- productName : String
Stores the product name.
- available : boolean
Indicates whether the product is available.
- observers : List<Observer>
Stores subscribed users.

Methods

- `subscribe(observer : Observer)`
Adds a user to the observer list.
 - `unsubscribe(observer : Observer)`
Removes a user from the observer list.
 - `notifyObservers()`
Notifies all users about availability changes.
 - `setAvailability(availability : boolean)`
Changes availability status and triggers notifications.
-

Observer (Interface)

Defines the update method for all observers.

Method

- `update(productName : String, available : boolean)`
Receives notification from the product.
-

User (Concrete Observer)

Represents a user subscribed to product updates.

Attribute

- `name : String`
Stores the user's name.

Method

- `update(productName : String, available : boolean)`
Displays notification when product availability changes.
-

6. Relationships Between Classes

Relationship

Description

Product implements Subject Product follows the Subject interface

Relationship	Description
User implements Observer	User follows the Observer interface
Product → Observer	Product communicates with observers

7. System Workflow

1. A user subscribes to a product.
 2. The product stores the user in the observer list.
 3. The product availability changes.
 4. `notifyObservers()` is triggered.
 5. All subscribed users receive updates through `update()`.
-

8. Advantages of Using Observer Pattern

- Loose coupling between objects.
 - Easier to add new observers.
 - Automatic real-time updates.
 - Better scalability and maintainability.
-

9. Example Scenario

- Product: "Laptop"
- Users: Ahmad and Sara

Steps

1. Ahmad and Sara subscribe to the Laptop product.
2. The product becomes available.
3. The system automatically notifies both users.

Example Output

Ahmad received notification:

Laptop is now available.

Sara received notification:

Laptop is now available.

10. Conclusion

The Observer Design Pattern provides an efficient way to implement notification systems. In this project, users automatically receive updates whenever product availability changes, making the system more dynamic, flexible, and easier to maintain.